Übungsblatt 9: Paralleles Rechnen & Multicore

Mikrocomputertechnik 3 · 32 Orientierungspunkte · ca. 75–90 Minuten

Musterlösung

Aufgabe 1: MESI mit drei Kernen (6 Punkte)

Drei Write-Back-Caches, anfangs alle I, unterer Speicher L enthält vier. Reads ohne weitere Kopie ergeben E. Fremdread auf M schreibt hier nach L zurück, alle beteiligten Kopien werden S. Vor M-Auswurf wird zurückgeschrieben.

1. Ergänzen Sie Zustände samt Werten und L nach jeder Aktion. (3 P)
2. Welchen unteren Wert darf es nach dem ersten Write geben? Welchen Wert muss der folgende fremde Read liefern? (1 P)
3. Welche fremden Kopien muss Kern 0 bei seinem späteren Write ungültig machen? (1 P)
4. Zwei S-Kopien bleiben übrig, eine wird sauber ausgeworfen. Muss die verbleibende automatisch E werden? (1 P)

Aktion	Kern 0	Kern 1	Kern 2	L
Kern 2 liest				
Kern 2 schreibt 9				
Kern 0 liest				
Kern 1 liest				
Kern 0 schreibt 13				
Kern 0 wirft aus				
Kern 1 liest				

Musterlösung 1

Aktion	Kern 0	Kern 1	Kern 2	L
Read 2	I	I	E:4	4
Write 2	I	I	M:9	4
Read 0	S:9	I	S:9	9
Read 1	S:9	S:9	S:9	9
Write 0	M:13	I	I	9
Auswurf 0	I	I	I	13
Read 1	I	E:13	I	13

Tabelle 3 P. L darf zunächst vier bleiben, der fremde Read muss neun erhalten. (1 P) Vor abgeschlossenem Write 0 werden die S-Kopien von Kern eins und zwei ungültig. (1 P) S darf nach sauberem Fremdauswurf S bleiben; ein automatischer E-Wechsel folgt nicht. (1 P)

Aufgabe 2: Acht-Byte-Objekte in einer großen Zeile (4 Punkte)

128-Byte-Cachezeilen, Basis `0x30004000`. Kern 0 schreibt nur a bei Offset acht, Kern 1 nur b bei Offset 24; beide Objekte sind acht Byte groß und getrennt zugeteilt. Anfangs besitzt Kern 1 die Zeile M.

1. Geben Sie Objektadressen und gemeinsamen vollständigen Zeilenbereich an. (1 P)
2. Schreibfolge 1, 0, 0, 1, 0, 1: Wie oft wechselt der Besitzer? (1 P)
3. b wird bei Offset 128 platziert. Neue Adresse? Warum reichte Basisausrichtung allein nicht? (1 P)
4. Repariert das auch verlorene Updates auf einem gemeinsam ungeschützten Zähler? (1 P)

Musterlösung 2

1. a `0x30004008`, b `0x30004018`, Zeile `0x30004000` bis `0x3000407F`. (1 P)
2. Vier Wechsel; wiederholte Writes des aktuellen Besitzers brauchen keinen Wechsel. (1 P)
3. b `0x30004080`. Auch der Objektabstand muss die Zeilentrennung herstellen. (1 P)
4. Nein. Derselbe Zähler braucht weiterhin atomare RMW oder passende Sperre. (1 P)

Aufgabe 3: LR/SC erhöht einen Wert um zwei (6 Punkte)

1. Hardwaremodell: Zwei Harts lesen jeweils Zähler zehn, addieren lokal zwei und schreiben beide zurück. Notieren Sie eine Folge und den Endwert statt des erwarteten 14. (2 P)
2. Wie unterscheidet sich ein gewöhnlicher ungeschützter C-Zähler von diesem vorgegebenen Hardwareablauf? Genügen getrennte atomare Loads und Stores? (1 P)
3. LR/SC soll dieselbe Addition atomar ausführen. Beide LR lesen anfangs zehn. SC von Hart 1 mit zwölf gelingt, SC von Hart 0 mit zwölf scheitert. Hart 0 wiederholt LR, berechnet neu und sein nächster SC gelingt. Geben Sie gelesene Werte, geschriebene Werte, SC-Status und Endwert an. (2 P)
4. Warum darf Hart 0 nicht einfach den alten Wert zwölf erneut speichern? Beweist jeder SC-Fehler einen fremden Store? (1 P)

Musterlösung 3

1. Load 0 zehn, Load 1 zehn, beide Register auf zwölf erhöhen, Store 1 zwölf, Store 0 zwölf: Endwert zwölf statt vierzehn. (2 P)
2. Ungeschützter konfliktierender C-Zugriff: Data Race, undefiniertes Verhalten. Atomare Einzelzugriffe können weiterhin die logische Gruppenwirkung verlieren. (1 P)
3. Beide LR liefern zehn. SC 1 schreibt zwölf, Status null; SC 0 schreibt nichts, Status ungleich null. Neuer LR 0 liefert zwölf; Addition ergibt vierzehn; erfolgreicher SC schreibt vierzehn mit Status null. (2 P)
4. Wiederholung muss vom neu gelesenen Wert ausgehen. SC kann auch ohne fremden Store scheitern. (1 P)

Aufgabe 4: Übergabe und ein dreiteiliger Wartekreis (6 Punkte)

1. Produzent schreibt einmalig gewöhnliche data 73 und danach atomar ready sieben mit Release. Konsument liest genau diese sieben mit Acquire und danach data; keine späteren Überschreibungen. Ergebnis und Synchronisationsvoraussetzung? (2 P)
2. Separat sind beide Objekte atomar und alle Zugriffe relaxed; data beginnt bei -1. Ist ready sieben mit data -1 erlaubt? Was wäre bei gewöhnlicher ungeschützter Payload anders? (1 P)
3. A hält X und wartet auf Y, B hält Y und wartet auf Z, C hält Z und wartet auf X. Zeichnen Sie den Kreis und nennen Sie eine globale Erwerbsreihenfolge. Darf ein Timeout fremde Sperren löschen? (2 P)
4. Ein Treiber hält P und wird unterbrochen; seine ISR wartet auf P. Warum genügt eine atomare Lockoperation nicht? Hält lokale Interruptsperre andere Harts fern? (1 P)

Musterlösung 4

1. Data 73; passendes Acquire muss den Release-Wert desselben Flags lesen, Payload davor schreiben und während dieser Übergabe unverändert lassen. (2 P)
2. Ja, ohne übergreifende Synchronisation. Gewöhnliche konfliktierende Payload hätte dagegen einen C-Data-Race mit undefiniertem Verhalten. (1 P)
3. $A \to Y \to B \to Z \to C \to X \to A$. Alle erwerben X vor Y vor Z, auch bei nur zwei benötigten Locks. Timeout berechtigt nicht zum Löschen fremder noch genutzter Sperren. (2 P)
4. Der unterbrochene Besitzer kann vor ISR-Rückkehr nicht freigeben. Atomizität löst den Selbstdeadlock nicht; lokale Sperre betrifft keine anderen Harts. (1 P)

Aufgabe 5: Drei Kerne und ungleiche Teilpakete (4 Punkte)

Ein-Kern-Zeit 150 Einheiten, serieller Anteil 0,3, paralleler Anteil 0,7. Zunächst perfekte Verteilung ohne Zusatzaufwand. $S(N) = 1/(s + (1-s)/N)$, Effizienz $E = S/N$.

1. Zeit, Speedup und Effizienz bei drei Kernen? (1 P)
2. Zeit, Speedup, Effizienz bei sechzig Kernen und theoretische Speedupgrenze? (1 P)
3. Bei drei Kernen kommen sechs Zeiteinheiten hinzu. Neue Zeit, Speedup und normierter Zusatz $h(3)$? (1 P)
4. Ohne diesen Zusatz: Die 105 parallelen Einheiten sind als 15, 25, 65 verteilt. Parallele Abschlusszeit, Gesamtzeit und Speedup? (1 P)

Musterlösung 5

1. $45 + 105/3 = 80$, Speedup 1,875, Effizienz 62,5 Prozent. (1 P)
2. $T = 46{,}75$, $S \approx 3{,}20856$, $E \approx 5{,}3476\,\%$, Grenzfaktor $1/0{,}3 \approx 3{,}333$. (1 P)
3. Zeit 86, Speedup $150/86 \approx 1{,}74419$, $h(3) = 6/150 = 0{,}04$. (1 P)
4. Paralleler Abschluss 65, gesamt 110, Speedup $150/110 \approx 1{,}36364$. (1 P)

Aufgabe 6: Ergebnisse veröffentlichen und übernehmen (6 Punkte)

C11-Lehrmodell: Zwei Worker lesen dieselbe unveränderliche Eingabe `[3, 6, 9, 12]`. Worker 0 addiert die Elemente mit Index null und eins, Worker 1 die mit Index zwei und drei. Jeder schreibt genau einmal ausschließlich seinen eigenen Eintrag im gewöhnlichen `int32_t teil[2]`. Die atomaren Meldungen `atomic_uint fertig[2]` beginnen bei null. Ein separater Sammler liest die Ergebnisse erst, wenn beide Meldungen eins sind. Alle Objekte sind vor dem Start initialisiert; kein weiterer Synchronisationsweg und kein Wiederverwenden der Meldungen. Die Werte laufen nicht über.

Die C11-Ausschnitte gehören zu verschiedenen Ausführungsfäden:

```
/* Worker i, i ist 0 oder 1: */
teil[i] = lokale_summe;
atomic_store_explicit(&fertig[i], 1u, memory_order_relaxed);

/* Sammler, gesamt wird nur von ihm verwendet: */
if (atomic_load_explicit(&fertig[0], memory_order_acquire) == 1u &&
    atomic_load_explicit(&fertig[1], memory_order_acquire) == 1u)
    gesamt = teil[0] + teil[1];
```

1. Welche Teilsummen und welcher Gesamtwert sind beabsichtigt? (1 P)
2. Beurteilen und reparieren Sie die Veröffentlichung. Zeichnen Sie die nötige Ordnungsbeziehung von jedem Worker bis zum Lesen seines Ergebnisses. Darf dem fehlerhaften C11-Programm eine bestimmte Liste möglicher Gesamtwerte zugeschrieben werden? (2 P)
3. Separates Layoutmodell: 64-Byte-Cachezeilen, zwei Vier-Byte-Ergebnisse bei `0x20002000` und `0x20002004`; die Kontrollfelder liegen in anderen Zeilen. Welche zusätzliche Kopienkoordination kann trotz verschiedener Ergebnisobjekte entstehen? Wählen Sie eine passende nächste Zeilenadresse für das zweite Objekt. Ersetzt diese Trennung die Reparatur aus Teil 2? (2 P)
4. Worker 1 erreicht seine Meldung nie. Garantiert eine korrekt geordnete Veröffentlichung trotzdem die vollständige Ergebnisübernahme durch den Sammler? Welche zusätzliche Entwurfsentscheidung fehlt? (1 P)

Musterlösung 6

1. Worker 0 liefert $3 + 6 = 9$, Worker 1 $9 + 12 = 21$, insgesamt $9 + 21 = 30$. Die Teilsummenberechnung verwendet getrennte lokale Akkumulatoren; gemeinsam verwendet werden erst die Ergebnisplätze. (1 P)
2. Der `relaxed`-Store macht die Meldung atomar, stellt aber keine Synchronisationsbeziehung zu den gewöhnlichen Ergebniszugriffen her. Jeder Worker schreibt zuerst seinen Ergebnisplatz und veröffentlicht danach mit `memory_order_release`. Der Sammler behält seine Acquire-Loads bei und liest beide Ergebnisplätze erst nach zwei beobachteten Einsen. (1 P) Je Worker gilt: Ergebnis-Store vor Release-Store; dieser synchronisiert mit dem Acquire-Load, der seine einmalige Eins liest; danach folgt der Ergebnis-Load. So entsteht die benötigte Happens-before-Beziehung. Ohne diese oder eine andere passende Synchronisation liegt beim konkurrierenden Zugriff auf den gewöhnlichen Ergebnisplatz ein C11-Data-Race mit undefiniertem Verhalten vor. Eine feste C-Endwertliste darf daraus nicht abgeleitet werden. (1 P)
3. Beide Ergebnisobjekte liegen in derselben Zeile: unabhängig beschriebene Objekte können False Sharing und zusätzliche Übernahmen des Zeilenschreibrechts verursachen. Eine tatsächliche Laufzeitstrafe ist damit noch nicht gemessen. (1 P) Das zweite Vier-Byte-Objekt

kann bei `0x20002040` liegen; die erste Adresse ist bereits auf 64 Byte ausgerichtet. Getrennte Zeilen ersetzen weder Atomizität noch die Release-/Acquire-Ordnung der Veröffentlichung. (1 P)

4. Nein. Ordnung garantiert keinen Fortschritt und keine beschränkte Wartezeit. Benötigt wird eine vereinbarte Fehler-, Abbruch- oder Überwachungspolitik für ausbleibende Worker; ein Timeout bräuchte zusätzlich ein Zeitmodell. (1 P)

Optional: am Rechner prüfen

Erweitern Sie `src/lab09_atomics.c` um drei Worker mit je 7000 atomaren Additionen: Der Zähler beginnt bei null und muss nach allen Joins 21000 enthalten. Prüfen Sie außerdem die einmalige Veröffentlichung von Payload 73. Führen Sie kein gewöhnliches ungeschütztes C-Race als Zahlenbeweis aus.